handling writes

what about writing to the cache?

two decision points:

if the value is not in cache, do we add it?

- if yes: need to load rest of block — *write-allocate*
- if no: missing out on locality? *write-no-allocate*

if value is in cache, when do we update next level?

- if immediately: extra writing *write-through*
- if later: need to remember to do so *write-back*

allocate on write?

say processor writes *less than whole* cache block

and block not yet in cache

two options:

write-allocate

fetch rest of cache block, replace written part

(then follow write-through or write-back policy)

write-no-allocate

don't use cache at all (send write to memory *instead*)

guess: not read soon?

allocate on write?

say processor writes *less than whole* cache block

and block not yet in cache

two options:

write-allocate

fetch *rest of cache block*, replace written part
(then follow write-through or write-back policy)

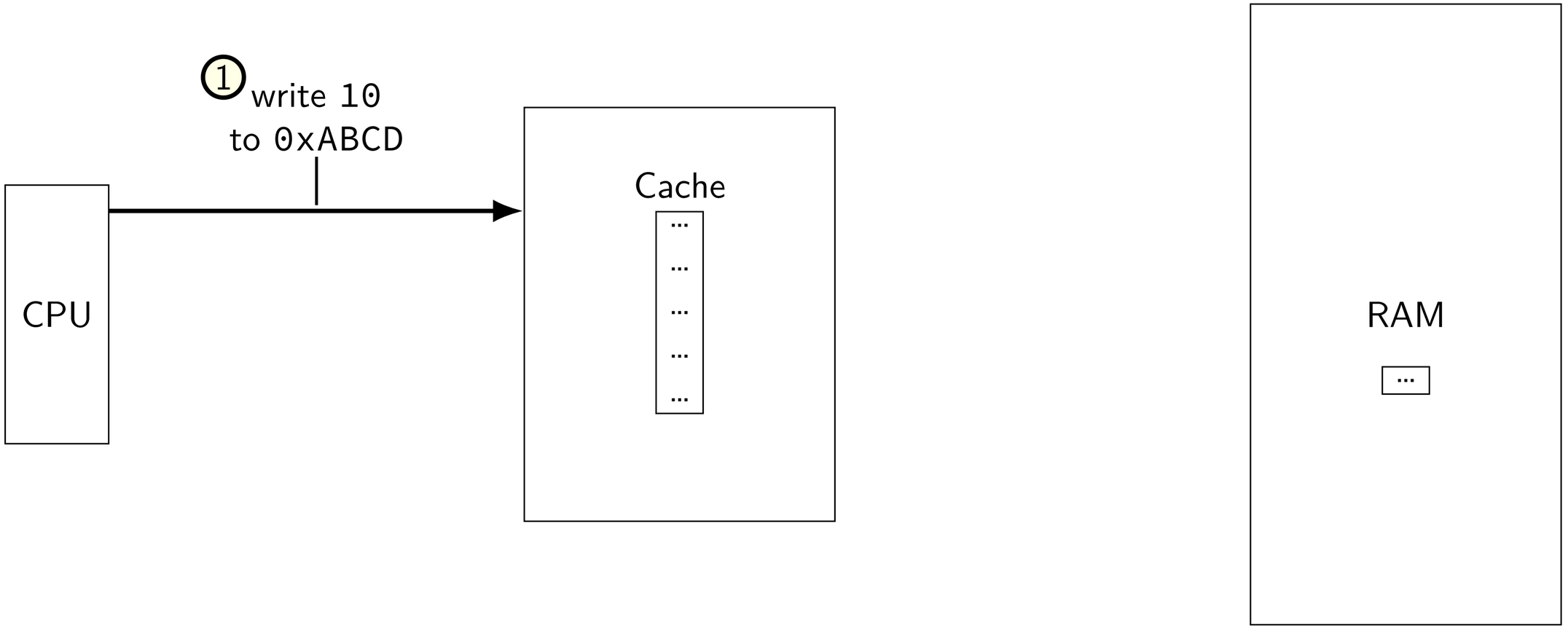
write-no-allocate

don't use cache at all (send write to memory *instead*)
guess: not read soon?

write-allocate v. write-no-allocate

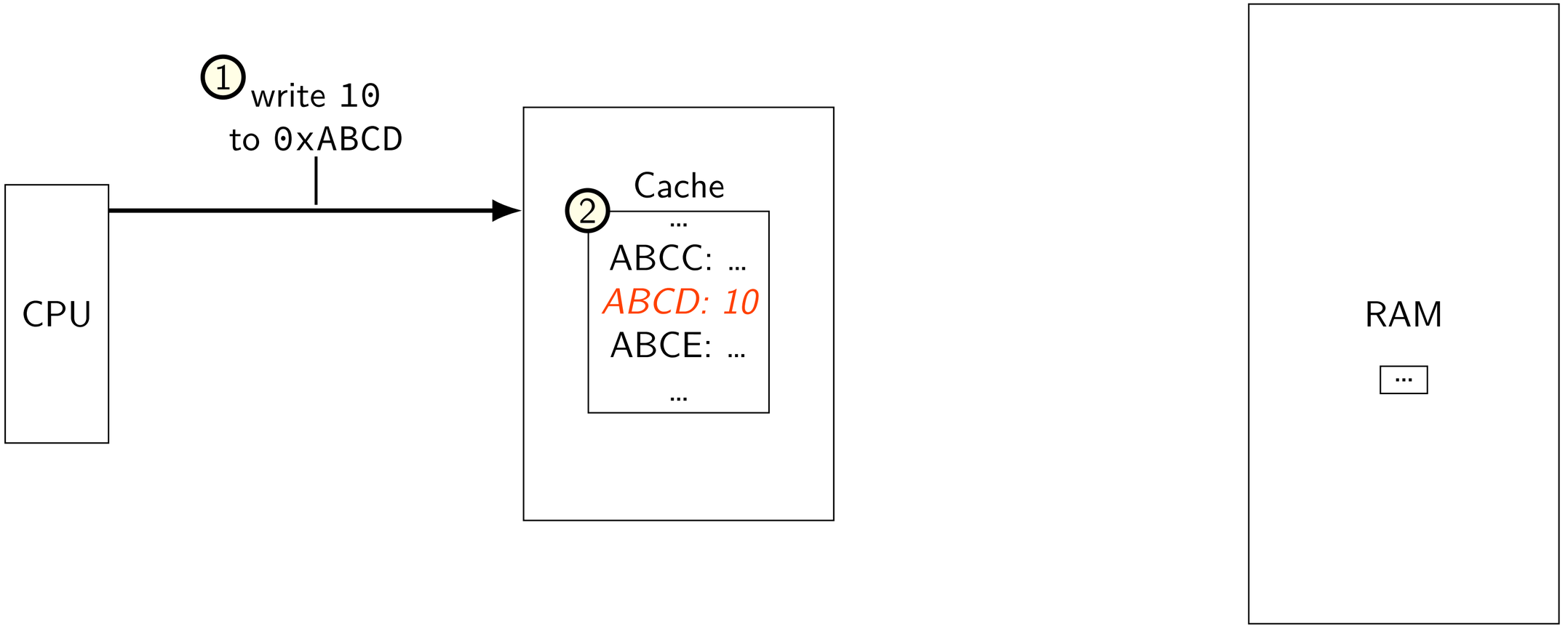
write-allocate v. write-no-allocate

option 1: write-allocate



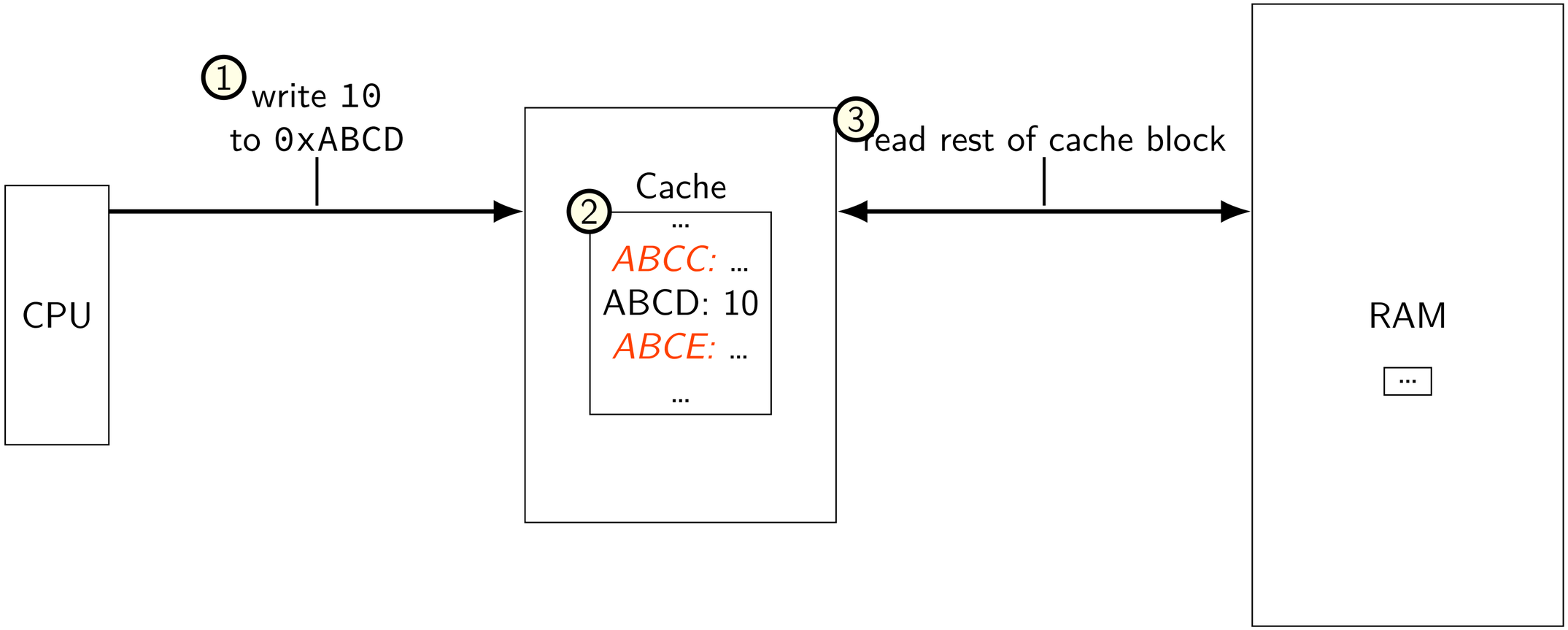
write-allocate v. write-no-allocate

option 1: write-allocate



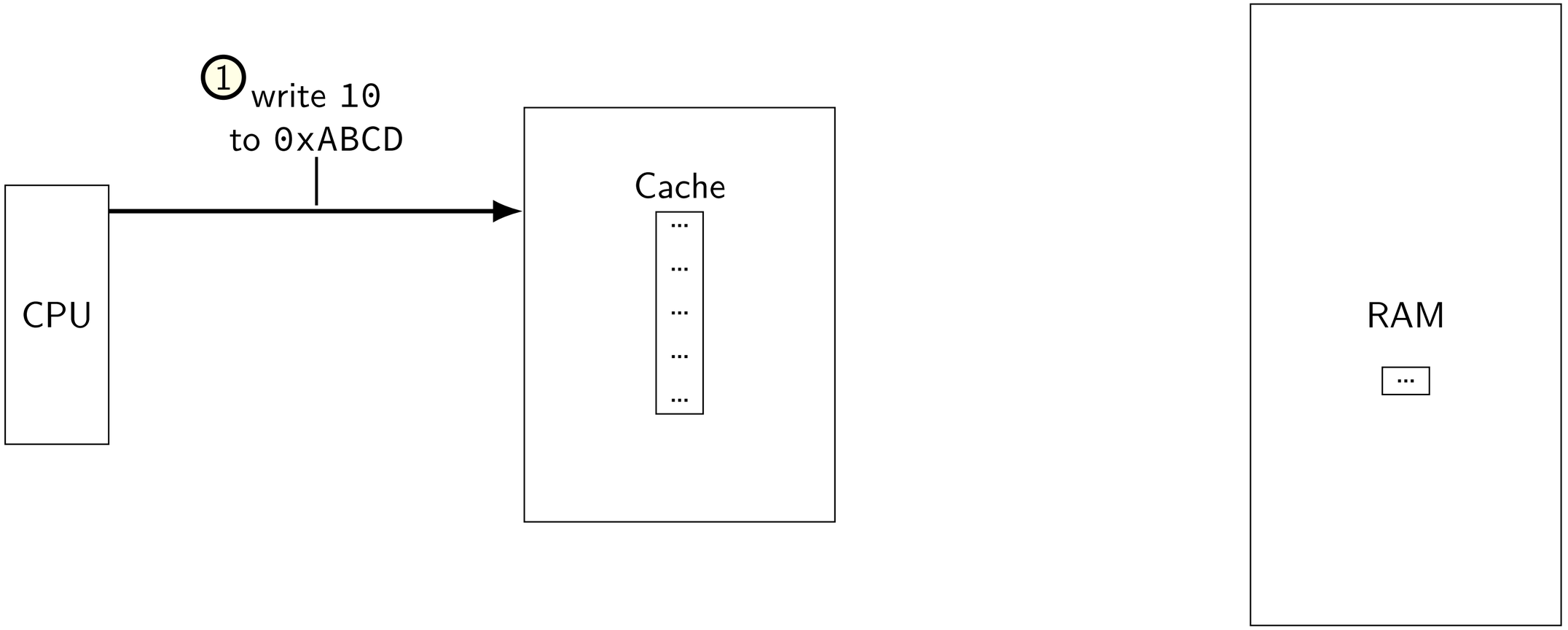
write-allocate v. write-no-allocate

option 1: write-allocate



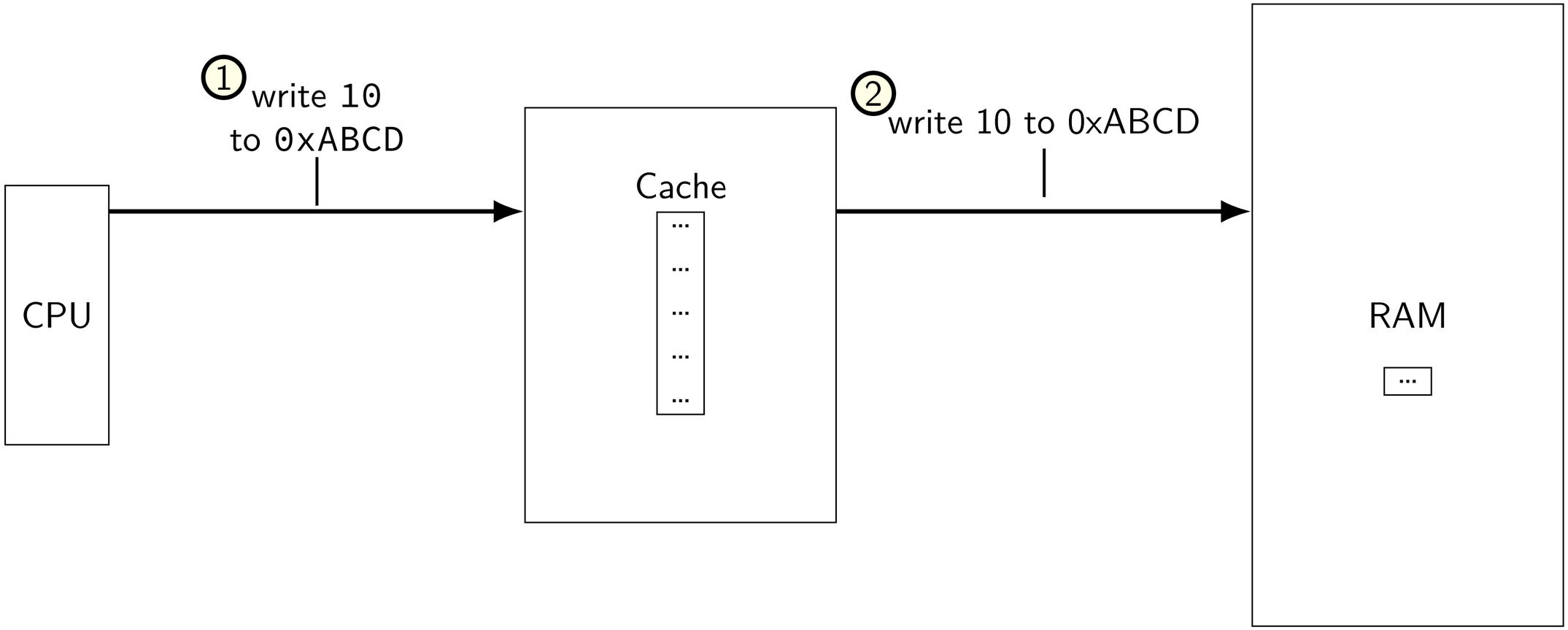
write-allocate v. write-no-allocate

option 2: write-no-allocate



write-allocate v. write-no-allocate

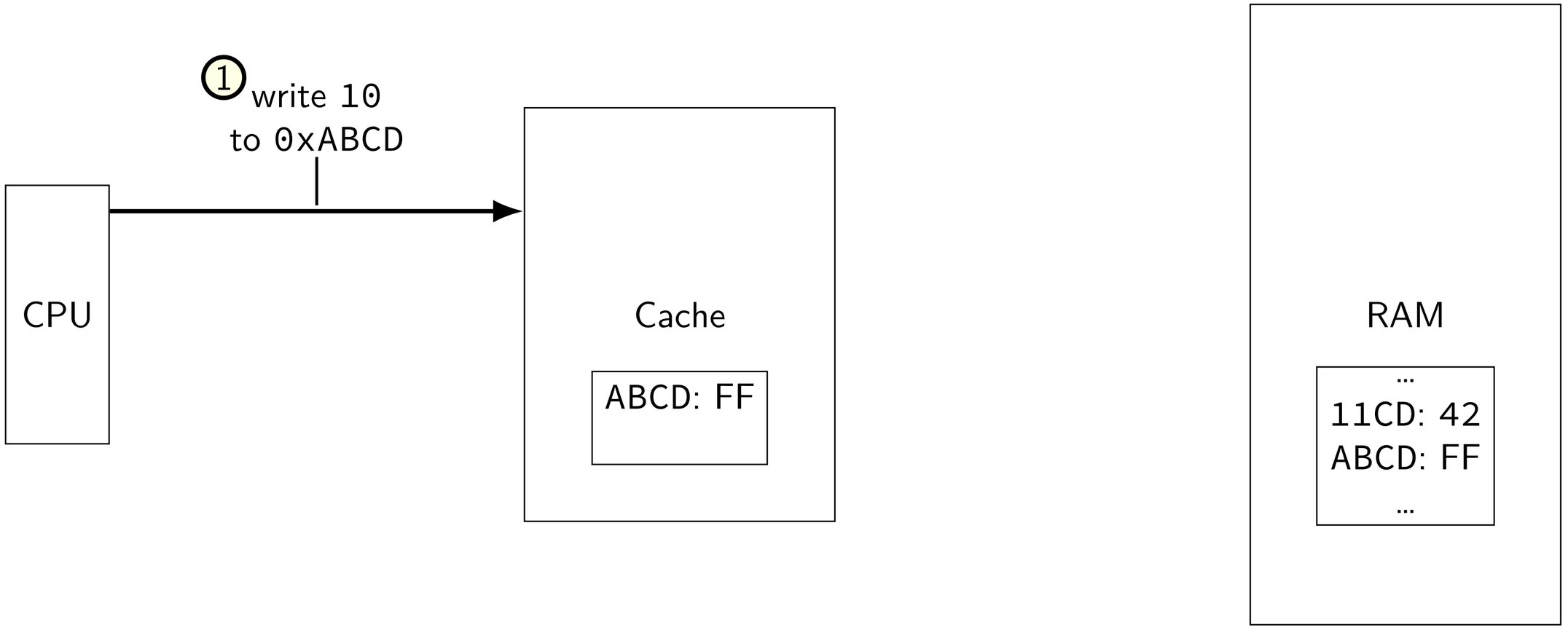
option 2: write-no-allocate



write-through v. write-back

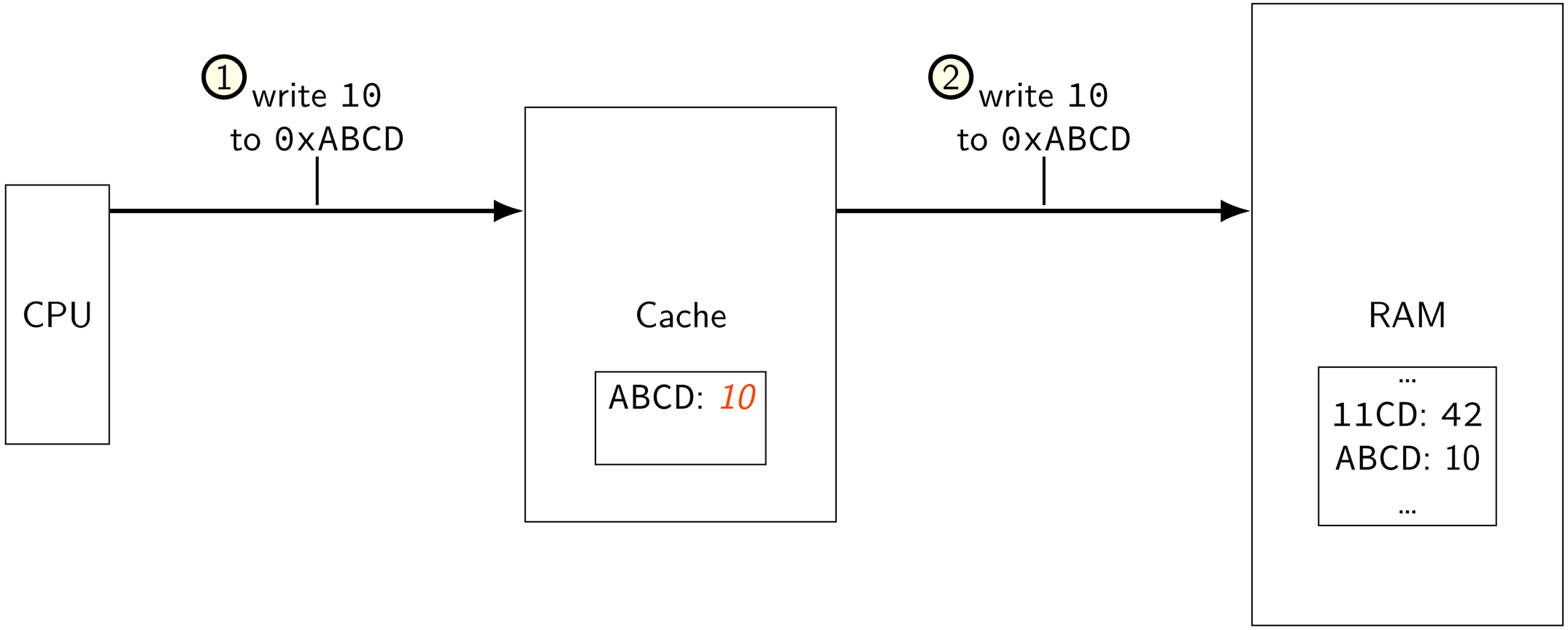
write-through v. write-back

option 1: write-through



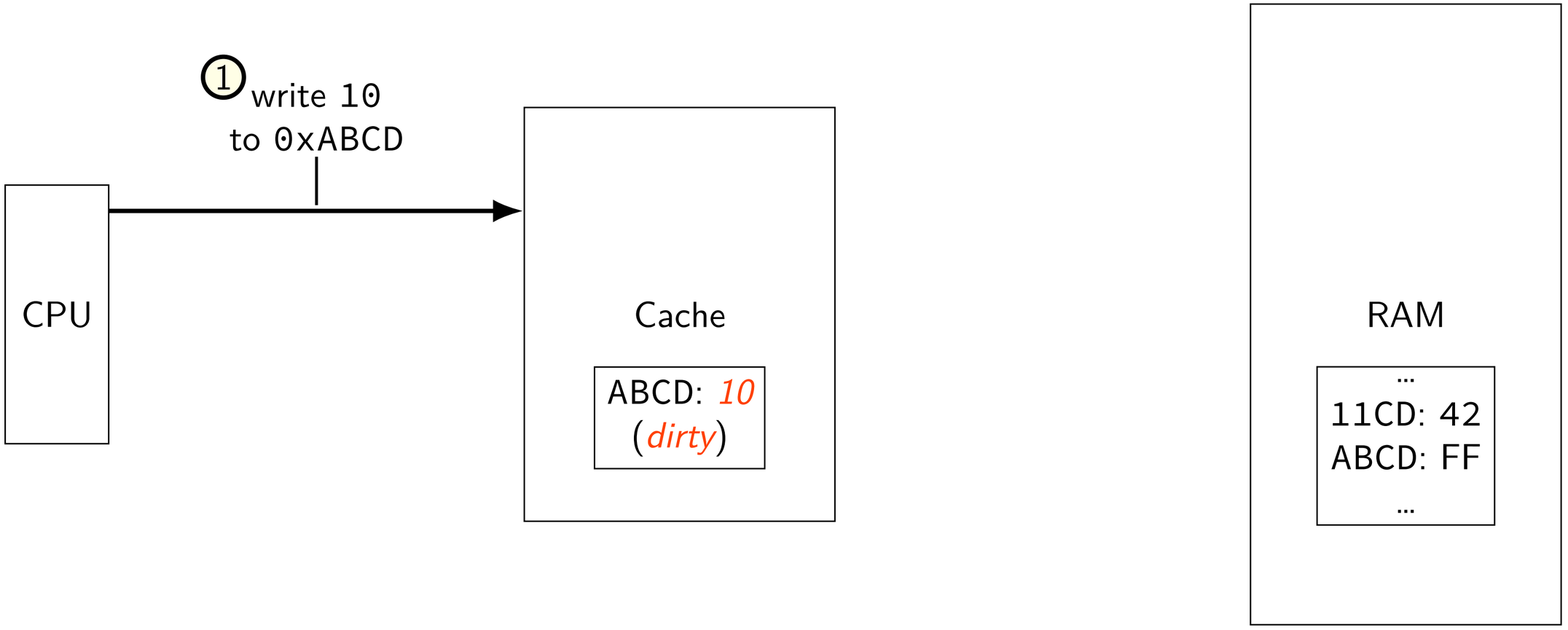
write-through v. write-back

option 1: write-through



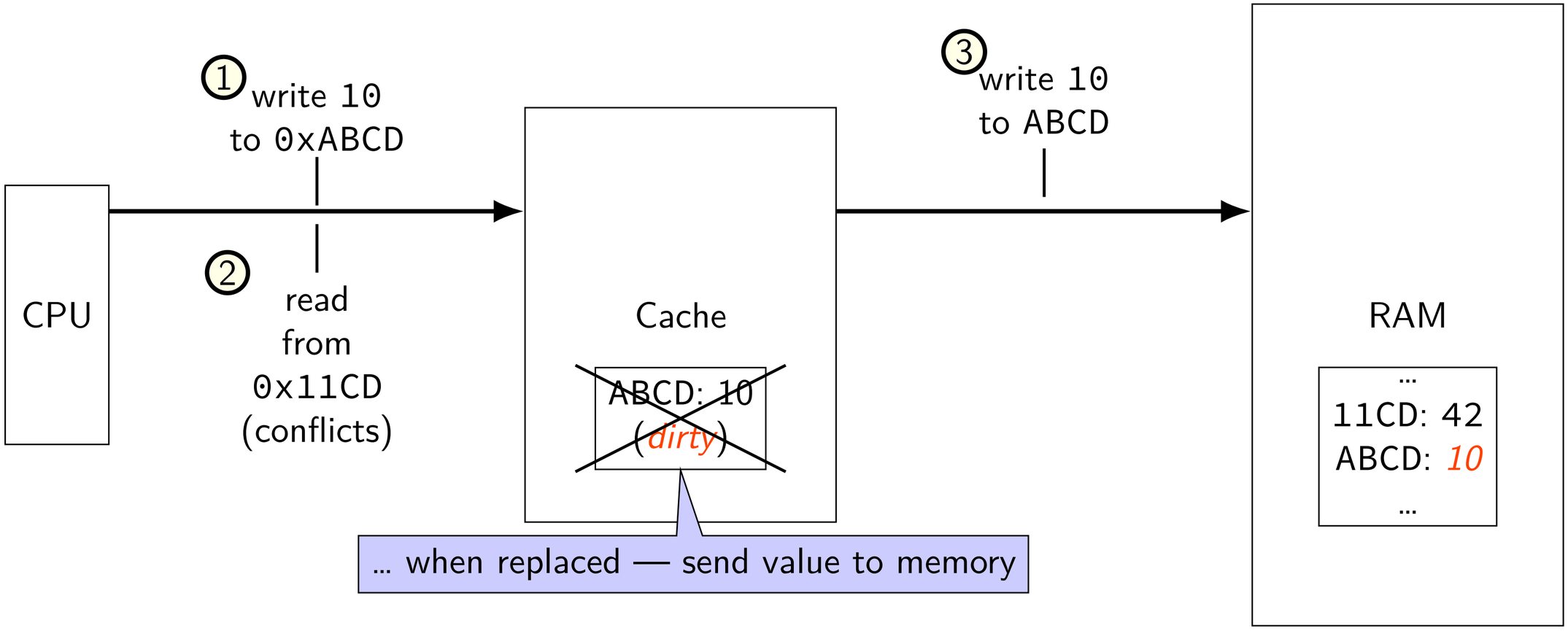
write-through v. write-back

option 2: write-back



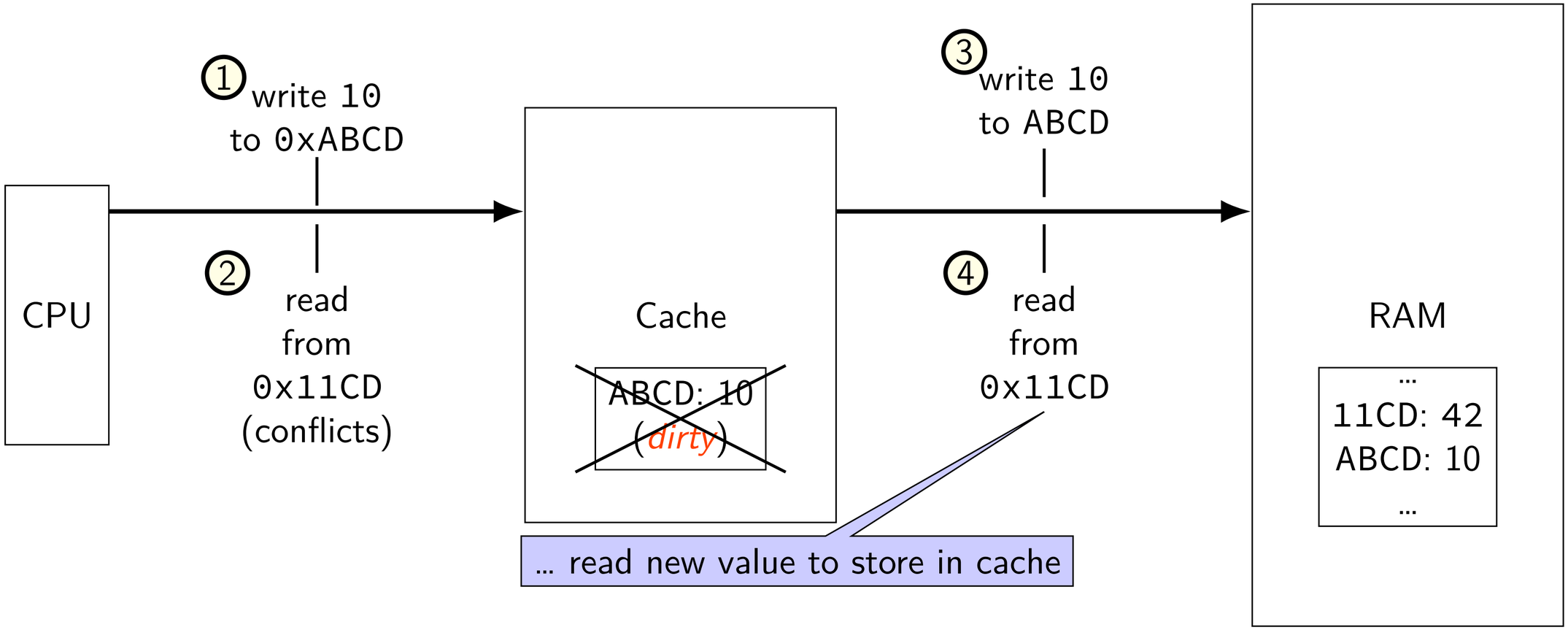
write-through v. write-back

option 2: write-back



write-through v. write-back

option 2: write-back



write-back policy

changed value!

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	000000	mem[0x00] mem[0x01]	0	1	011000	mem[0x60]* mem[0x61]*	1	1
1	1	011000	mem[0x62] mem[0x63]	0	0				0

1 = dirty (different than memory)
needs to be written if evicted

write-allocate + write-back

write-allocate + write-back

2-way set associative, LRU, writeback

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	000000	mem[0x00] mem[0x01]	0	1	011000	mem[0x60]* mem[0x61]*	1	1
1	1	011000	mem[0x62] mem[0x63]	0	0				0

writing 0xFF into address 0x04?

index 0, tag 000001

write-allocate + write-back

2-way set associative, LRU, writeback

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	000000	mem[0x00] mem[0x01]	0	1	011000	mem[0x60]* mem[0x61]*	1	1
1	1	011000	mem[0x62] mem[0x63]	0	0				0

writing 0xFF into address 0x04?

index 0, tag 000001

step 1: find *least recently used* block

write-allocate + write-back

2-way set associative, LRU, writeback

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	000000	mem[0x00] mem[0x01]	0	1	011000	mem[0x60]* mem[0x61]*	~~1~~	1
1	1	011000	mem[0x62] mem[0x63]	0	0				0

writing 0xFF into address 0x04?

index 0, tag 000001

step 1: find *least recently used* block

step 2: possibly writeback old block

write-allocate + write-back

2-way set associative, LRU, writeback

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	000000	mem[0x00] mem[0x01]	0	1	000001	0xFF mem[0x05]	1	*0*
1	1	011000	mem[0x62] mem[0x63]	0	0				0

writing 0xFF into address 0x04?

index 0, tag 000001

step 1: find *least recently used* block

step 2: possibly writeback old block

step 3a: read in new block – to get mem[0x05]

step 3b: update LRU information

write-no-allocate + write-back

2-way set associative, LRU, writeback

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	000000	mem[0x00] mem[0x01]	0	1	011000	mem[0x60]* mem[0x61]*	1	1
1	1	011000	mem[0x62] mem[0x63]	0	0				0

writing 0xFF into address 0x04?

step 1: is it in cache yet?

step 2: no, *just send it to memory*

cache write exercise (1)

2-way set associative, LRU, *write-allocate, writeback*

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	001100	mem[0x30] mem[0x31]	0	1	010000	mem[0x40]* mem[0x41]*	1	0
1	1	011000	mem[0x62] mem[0x63]	0	1	001100	mem[0x32]* mem[0x33]*	1	1

for each of the following accesses, performed alone, would it require (a) reading a value from memory (or next level of cache) and (b) writing a value to the memory (or next level of cache)?

writing 1 byte to 0x33

reading 1 byte from 0x52

reading 1 byte from 0x50

cache write exercise (1, solution)

2-way set associative, LRU, *write-allocate, writeback*

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	001100	mem[0x30] mem[0x31]	0	1	010000	mem[0x40]* mem[0x41]*	1	0
1	1	011000	mem[0x62] mem[0x63]	0	1	001100	mem[0x32]* mem[0x33]*	1	1

writing 1 byte to 0x33:

reading 1 byte from 0x52:

reading 1 byte from 0x50:

cache write exercise (1, solution)

2-way set associative, LRU, *write-allocate, writeback*

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	001100	mem[0x30] mem[0x31]	0	1	010000	mem[0x40]* mem[0x41]*	1	0
1	1	011000	mem[0x62] mem[0x63]	0	1	001100	mem[0x32]* mem[0x33]*	1	1

writing 1 byte to 0x33:

reading 1 byte from 0x52:

reading 1 byte from 0x50:

cache write exercise (1, solution)

2-way set associative, LRU, *write-allocate, writeback*

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	001100	mem[0x30] mem[0x31]	0	1	010000	mem[0x40]* mem[0x41]*	1	0
1	1	011000	mem[0x62] mem[0x63]	0	1	001100	mem[0x32]* mem[0x33]*	1	~~1~~0

writing 1 byte to 0x33: (set 1, offset 1) no next-level read or write

reading 1 byte from 0x52:

reading 1 byte from 0x50:

cache write exercise (1, solution)

2-way set associative, LRU, *write-allocate, writeback*

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	001100	mem[0x30] mem[0x31]	0	1	010000	mem[0x40]* mem[0x41]*	1	0
1	1	011000	mem[0x62] mem[0x63]	0	1	001100	mem[0x32]* mem[0x33]*	1	1

writing 1 byte to 0x33: (set 1, offset 1) no next-level read or write

reading 1 byte from 0x52:

reading 1 byte from 0x50:

cache write exercise (1, solution)

2-way set associative, LRU, *write-allocate, writeback*

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	001100	mem[0x30] mem[0x31]	0	1	010000	mem[0x40]* mem[0x41]*	1	0
1	1	011000	mem[0x62] mem[0x63]	0	1	101000	mem[0x52] mem[0x53]	~~1~~0	~~1~~0

writing 1 byte to 0x33: (set 1, offset 1) no next-level read or write

reading 1 byte from 0x52: (set 1, offset 0) *write* back 0x32-0x33; *read* 0x52-0x53

reading 1 byte from 0x50:

cache write exercise (1, solution)

2-way set associative, LRU, *write-allocate, writeback*

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	001100	mem[0x30] mem[0x31]	0	1	010000	mem[0x40]* mem[0x41]*	1	0
1	1	011000	mem[0x62] mem[0x63]	0	1	001100	mem[0x32]* mem[0x33]*	1	1

writing 1 byte to 0x33: (set 1, offset 1) no next-level read or write

reading 1 byte from 0x52: (set 1, offset 0) *write* back 0x32-0x33; *read* 0x52-0x53

reading 1 byte from 0x50:

cache write exercise (1, solution)

2-way set associative, LRU, *write-allocate, writeback*

index	valid	tag	value	dirty	valid	tag	value	dirty	LRU
0	1	101000	mem[0x50] mem[0x51]	0	1	010000	mem[0x40]* mem[0x41]*	1	~~0~~1
1	1	011000	mem[0x62] mem[0x63]	0	1	001100	mem[0x32]* mem[0x33]*	1	1

writing 1 byte to 0x33: (set 1, offset 1) no next-level read or write

reading 1 byte from 0x52: (set 1, offset 0) *write* back 0x32-0x33; *read* 0x52-0x53

reading 1 byte from 0x50: (set 0, offset 0) replace 0x30-0x31 (no write back); *read* 0x50-0x51

cache write exercise (2)

2-way set associative, LRU, *write-no-allocate, write-through*

index	valid	tag	value	valid	tag	value	LRU
0	1	001100	mem[0x30] mem[0x31]	1	010000	mem[0x40] mem[0x41]	0
1	1	011000	mem[0x62] mem[0x63]	1	001100	mem[0x32] mem[0x33]	1

for each of the following accesses, *performed alone*, would it require (a) reading a value from memory and (b) writing a value to the memory?

writing 1 byte to 0x33

reading 1 byte from 0x52

reading 1 byte from 0x50

cache write exercise (2, solution)

2-way set associative, LRU, *write-no-allocate, write-through*

index	valid	tag	value	valid	tag	value	LRU
0	1	001100	mem[0x30] mem[0x31]	1	010000	mem[0x40] mem[0x41]	0
1	1	011000	mem[0x62] mem[0x63]	1	001100	mem[0x32] mem[0x33]	1

writing 1 byte to 0x33:

reading 1 byte from 0x52:

reading 1 byte from 0x50:

cache write exercise (2, solution)

2-way set associative, LRU, *write-no-allocate, write-through*

index	valid	tag	value	valid	tag	value	LRU
0	1	001100	mem[0x30] mem[0x31]	1	010000	mem[0x40] mem[0x41]	0
1	1	011000	mem[0x62] mem[0x63]	1	001100	mem[0x32] mem[0x33]	~~1~~0

writing 1 byte to 0x33:

reading 1 byte from 0x52:

reading 1 byte from 0x50:

cache write exercise (2, solution)

2-way set associative, LRU, *write-no-allocate, write-through*

index	valid	tag	value	valid	tag	value	LRU
0	1	001100	mem[0x30] mem[0x31]	1	010000	mem[0x40] mem[0x41]	0
1	1	011000	mem[0x62] mem[0x63]	1	001100	mem[0x32] mem[0x33]	1

writing 1 byte to 0x33: (set 1, offset 1) write-through 0x33 modification

reading 1 byte from 0x52:

reading 1 byte from 0x50:

cache write exercise (2, solution)

2-way set associative, LRU, *write-no-allocate, write-through*

index	valid	tag	value	valid	tag	value	LRU
0	1	001100	mem[0x30] mem[0x31]	1	010000	mem[0x40] mem[0x41]	0
1	1	011000	mem[0x62] mem[0x63]	1	101000	mem[0x52] mem[0x53]	~~1~~0

writing 1 byte to 0x33: (set 1, offset 1) write-through 0x33 modification

reading 1 byte from 0x52: (set 1, offset 0) replace 0x32-0x33; *read* 0x52-0x53

reading 1 byte from 0x50:

cache write exercise (2, solution)

2-way set associative, LRU, *write-no-allocate, write-through*

index	valid	tag	value	valid	tag	value	LRU
0	1	001100	mem[0x30] mem[0x31]	1	010000	mem[0x40] mem[0x41]	0
1	1	011000	mem[0x62] mem[0x63]	1	001100	mem[0x32] mem[0x33]	1

writing 1 byte to 0x33: (set 1, offset 1) write-through 0x33 modification

reading 1 byte from 0x52: (set 1, offset 0) replace 0x32-0x33; *read* 0x52-0x53

reading 1 byte from 0x50:

cache write exercise (2, solution)

2-way set associative, LRU, *write-no-allocate, write-through*

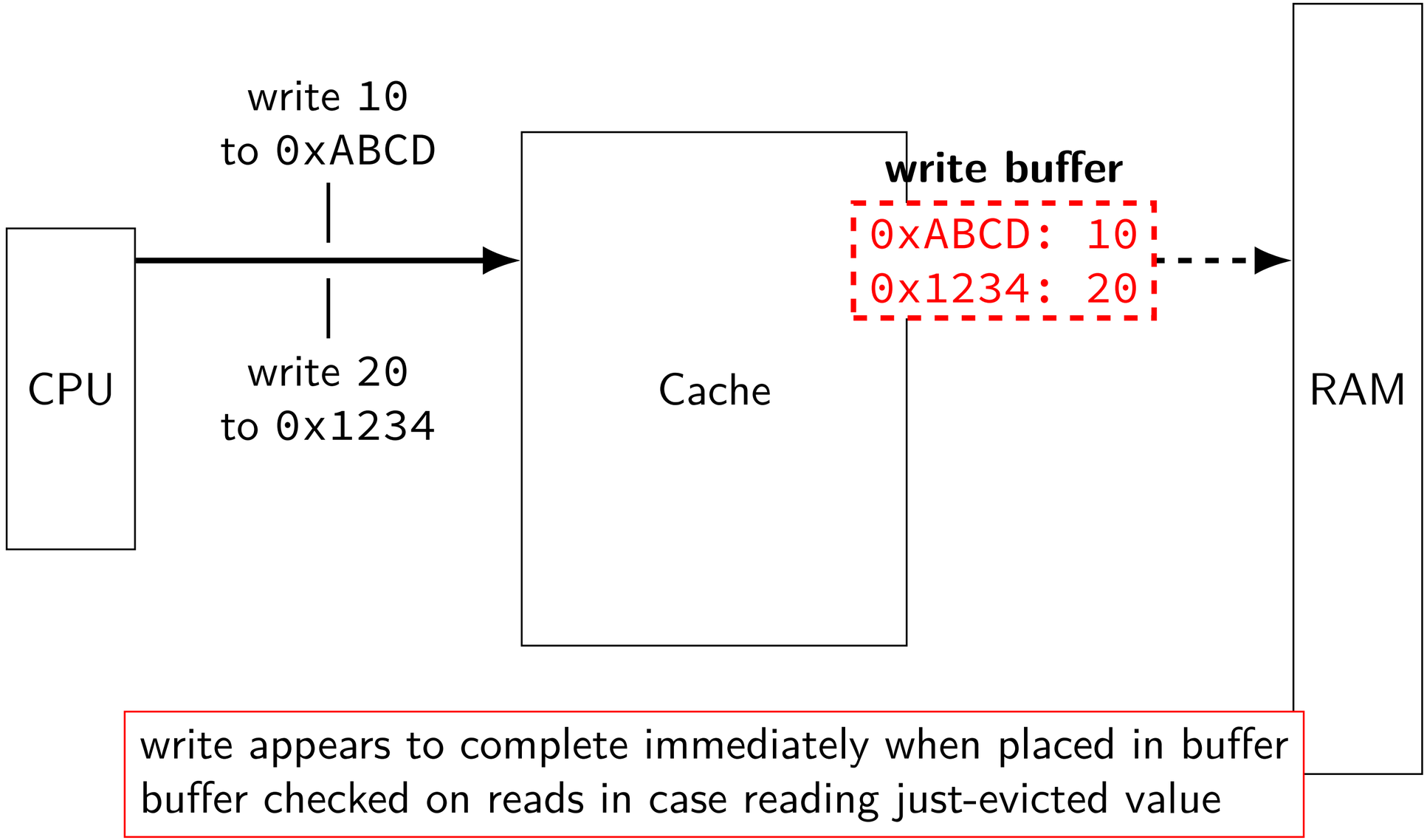
index	valid	tag	value	valid	tag	value	LRU
0	1	101000	mem[0x50] mem[0x51]	1	010000	mem[0x40] mem[0x41]	~~0~~1
1	1	011000	mem[0x62] mem[0x63]	1	001100	mem[0x32] mem[0x33]	1

writing 1 byte to 0x33: (set 1, offset 1) write-through 0x33 modification

reading 1 byte from 0x52: (set 1, offset 0) replace 0x32-0x33; *read* 0x52-0x53

reading 1 byte from 0x50: (set 0, offset 0) replace 0x30-0x31; *read* 0x50-0x51

fast writes with write-through caches



write appears to complete immediately when placed in buffer
buffer checked on reads in case reading just-evicted value

backup slides